計算機科学実験及演習 3 ハードウェア

SIMPLE/B プロセッサ設計レポート

基本アーキテクチャの実装と検証

題目	SIMPLE/B プロセッサの設計と実装
学籍番号	（学籍番号を記入）
氏名	（氏名を記入）
提出日	2026 年 5 月 8 日

京都大学 工学部情報学科 計算機科学コース

目次

1 **はじめに** 2

2 **実験環境** 2

3 **SIMPLE アーキテクチャ概要** 2
- 3.1 レジスタとメモリ 2
- 3.2 命令セットアーキテクチャ 2

4 **SIMPLE/B マイクロアーキテクチャ** 3
- 4.1 全体構成 3
- 4.2 フェーズの動作 4
- 4.3 データパスの選択 4
- 4.4 制御信号 5

5 **モジュール設計** 6
- 5.1 モジュール構成 6
- 5.2 ALU (`alu.v`) 6
- 5.3 シフタ (`shifter.v`) 8
- 5.4 レジスタファイル (`regfile.v`) 9
- 5.5 主記憶 (`mem.v`) 9
- 5.6 CPU コア (`simple_cpu.v`) 10
- 5.7 トップモジュール (`simple_top.v`) 15

6 **テストプログラム** 17
- 6.1 基本テスト: 加算と出力 17
- 6.2 応用テスト: 遅延付きカウンタ 17

7 **動作検証** 18
- 7.1 テストベンチ 18
- 7.2 シミュレーション結果 20
- 7.3 命令実行トレース 21

8 **性能評価** 21
- 8.1 命令実行サイクル数 21
- 8.2 回路規模の見積もり 21
- 8.3 最大動作周波数 22
- 8.4 改良の方向性 22

9 **まとめ** 22

1 はじめに

本レポートでは、SIMPLE (SIxteen-bit MicroProcessor for Laboratory Experiment) アーキテクチャに準拠したプロセッサ SIMPLE/B の設計と実装について報告する。SIMPLE/B は設計資料（ver 4.0）に記載された基本設計に基づく 5 フェーズ順次実行方式のプロセッサであり、Verilog HDL で記述し、Icarus Verilog によるシミュレーションで動作を検証した。

2 実験環境

表 1　実験環境

項目	内容
FPGA ボード	PowerMedusa MU500-RX/RK
FPGA	Altera Cyclone IV EP4CE30F23I7N
CAD ツール	Intel Quartus Prime 20.1 Standard Edition
シミュレータ	Icarus Verilog 11.0 / ModelSim-Intel FPGA Starter Edition
OS	Ubuntu 22.04 LTS (WSL2)
クロック	20MHz 発振器

3 SIMPLE アーキテクチャ概要

3.1 レジスタとメモリ

SIMPLE は 1 語 16 ビットのコンピュータであり、以下の資源を持つ。

- **主記憶**: 16 ビットアドレス、64KW（語単位）のアドレス空間。Cyclone IV EP4CE30 では RAM 容量の制限により最大 33KW。本実装では 256 語（8 ビットアドレス）の分散 RAM を使用。
- **汎用レジスタ**: 8 本（`r[0]`〜`r[7]`）、各 16 ビット。
- **プログラムカウンタ (PC)**: 実行中の命令のアドレスを保持。
- **条件コード**: S（符号）、Z（ゼロ）、C（桁上げ）、V（オーバーフロー）の 4 ビット。

3.2 命令セットアーキテクチャ

SIMPLE の命令は全て 1 語（16 ビット）の固定長であり、4 種類の命令形式を持つ。

3.2.1 演算/入出力命令

3.2.2 ロード/ストア命令

- **LD Ra, d(Rb)**: `r[Ra] = *(r[Rb] + sign_ext(d))` — レジスタ間接 + 変位

表 2　命令形式

形式	ビット配置	用途
(a) 演算/IO	`11|Rs(3)|Rd(3)|op3(4)|d(4)`	ALU, シフト, IN/OUT, HLT
(b) LD/ST	`op1(2)|Ra(3)|Rb(3)|d(8)`	メモリアクセス
(c) LI/B	`10|op2(3)|Rb(3)|d(8)`	即値ロード, 無条件分岐
(d) Bcc	`10|111|cond(3)|d(8)`	条件分岐

表 3　演算/入出力命令一覧

ニーモニック	op3	機能	フラグ設定
ADD Rd,Rs	0000	`r[Rd] = r[Rd] + r[Rs]`	S,Z,C,V
SUB Rd,Rs	0001	`r[Rd] = r[Rd] - r[Rs]`	S,Z,C,V
AND Rd,Rs	0010	`r[Rd] = r[Rd] & r[Rs]`	S,Z; C=0
OR Rd,Rs	0011	`r[Rd] = r[Rd] | r[Rs]`	S,Z; C=0
XOR Rd,Rs	0100	`r[Rd] = r[Rd] ^ r[Rs]`	S,Z; C=0
CMP Rd,Rs	0101	`r[Rd] - r[Rs]`（フラグのみ）	S,Z,C,V
MOV Rd,Rs	0110	`r[Rd] = r[Rs]`	S,Z; C=0
SLL Rd,d	1000	左論理シフト	S,Z,C; V=0
SLR Rd,d	1001	左循環シフト	S,Z; C=0, V=0
SRL Rd,d	1010	右論理シフト	S,Z,C; V=0
SRA Rd,d	1011	右算術シフト	S,Z,C; V=0
IN Rd	1100	`r[Rd] = input`	なし
OUT Rs	1101	`output = r[Rs]`	なし
HLT	1111	停止	なし

- **ST Ra, d(Rb)**: `*(r[Rb] + sign_ext(d)) = r[Ra]`

3.2.3　即値ロード/分岐命令

- **LI Rb, d**: `r[Rb] = sign_ext(d)` — 8 ビット即値の符号拡張
- **B d**: `PC = PC + 1 + sign_ext(d)` — PC 相対無条件分岐

3.2.4　条件分岐命令

4　SIMPLE/B マイクロアーキテクチャ

4.1　全体構成

SIMPLE/B は 1 命令を 5 つのフェーズに分割し、順次活性化しながら実行する。図 1 にブロック図、図 2 にフェーズのタイミングを示す。

表 4　条件分岐命令

ニーモニック	cond	分岐条件
BE d	000	$Z = 1$（等しい）
BLT d	001	$S \oplus V = 1$（符号付き小なり）
BLE d	010	$Z \vee (S \oplus V) = 1$（符号付き以下）
BNE d	011	$Z = 0$（等しくない）

4.2　フェーズの動作

1. **p1 — 命令フェッチ (IF)**: PC が保持するアドレスの命令を主記憶からフェッチし、IR (Instruction Register) に格納する。同時に PC を 1 加算する。
2. **p2 — レジスタ読み出し (RR)**: IR の `Rs/Ra` フィールド（`I[13:11]`）で指定されるレジスタの値を AR に、`Rd/Rb` フィールド（`I[10:8]`）で指定されるレジスタの値を BR に格納する。
3. **p3 — 演算 (EX)**: 命令に応じて ALU またはシフタで演算を行い、結果を DR (Data Register) に格納する。演算命令では条件コード S, Z, C, V を更新する。LD/ST 命令では BR + sign_ext(d) により実効アドレスを計算する。分岐命令では PC + sign_ext(d) により分岐先アドレスを計算する。
4. **p4 — 主記憶アクセス / 入出力 (MA)**: LD 命令では DR の値をアドレスとして主記憶を読み出し、MDR に格納する。ST 命令では AR の値（`r[Ra]`）を DR のアドレスに書き込む。IN 命令では外部入力を MDR に格納し、OUT 命令では AR の値を外部出力する。
5. **p5 — レジスタ書き込み (WB)**: DR または MDR の値を、命令で指定されたレジスタに書き込む。分岐命令では DR の値を PC に書き込む。HLT 命令では実行を停止する。

4.3　データパスの選択

各フェーズにおけるデータパスの選択を表 5 に示す。

表 5　命令種別ごとのデータパス

命令	p2		p3			p4	p5	
	AR	BR	ALU 入力 A	ALU 入力 B	結果	動作	書込先	データ
ALU	r[Rs]	r[Rd]	BR	AR	DR	—	r[Rd]	DR
CMP	r[Rs]	r[Rd]	BR	AR	—	—	—	—
Shift	—	r[Rd]	—	—	DR	—	r[Rd]	DR
LD	r[Ra]	r[Rb]	BR	sext(d)	DR	MDR←mem[DR]	r[Ra]	MDR
ST	r[Ra]	r[Rb]	BR	sext(d)	DR	mem[DR]←AR	—	—
LI	—	—	0	sext(d)	DR	—	r[Rb]	DR
B	—	—	PC	sext(d)	DR	—	PC	DR
Bcc	—	—	PC	sext(d)	DR	—	PC/—	DR
IN	—	—	—	—	—	MDR←input	r[Rd]	MDR
OUT	r[Rs]	—	—	—	—	output←AR	—	—

```
        +----------+
+------>|    PC     |------+----> アドレスバス
|       +----------+      |
|           |  +1         v
|           v          +------+
|  p1  +----------+     | 主記憶 |
+------|   IR     |<---|      |<--- データバス
|      +----------+     +------+
|         デコード          ^
|           |  |          |
|      p2   v  v          |
|      +----+----+        |
|      | AR | BR |        |
|      +----+----+        |
|        |    |           |
|   p3   v    v           |
|      +--------+  +-------+
|      |  ALU   |  |シフタ  |
|      +--------+  +-------+
|         |  |          |
|    p4   v  v          |
|      +------+ +-----+
|      | SZCV | | DR  |----+----> アドレスバス
|      +------+ +-----+    |
|                  |       |
|   p5          +------+   |
+---------------|  MDR |<--+<--- 外部入力
                +------+
                   |-----> 外部出力
```

図 1　SIMPLE/B ブロック図（簡略版）

4.4　制御信号

制御部はフェーズカウンタ（3 ビット、0〜4 を巡回）と命令デコード結果に基づき、以下の制御を行う。

- **メモリアドレス選択**: p1 では PC、p4 では DR
- **メモリ書き込み許可**: p4 かつ ST 命令のときのみ有効
- **レジスタ書き込み**: p5 かつ書き込みを伴う命令（ALU/Shift/LD/LI/IN）のとき有効。書き

```
clock  __/~~\__/~~\__/~~\__/~~\__/~~\__/~~\__
phase   | p1 | p2 | p3 | p4 | p5 | p1 | ...
        | IF | RR | EX | MA | WB | IF | ...
```

図 2　SIMPLE/B の 5 フェーズ実行

込み先は LD 命令では `I[13:11]`（Ra）、その他では `I[10:8]`（Rd/Rb）。

- **exec 制御**: exec の立ち上がりエッジで実行開始、HLT 命令で停止

5　モジュール設計

5.1　モジュール構成

表 6　モジュール一覧

ファイル	モジュール名	機能
`alu.v`	`alu`	算術論理演算ユニット。ADD/SUB/AND/OR/XOR/CMP/MOV の 7 演算と条件コード生成。
`shifter.v`	`shifter`	バレルシフタ。SLL/SLR/SRL/SRA の 4 種シフトと条件コード C 生成。
`regfile.v`	`regfile`	8×16 ビットレジスタファイル。非同期 2 リード、同期 1 ライト。
`simple_cpu.v`	`simple_cpu`	CPU コア。5 フェーズ FSM、命令デコード、データパス制御。
`mem.v`	`mem`	主記憶。非同期リード、同期ライト。`$readmemh` で初期化。
`simple_top.v`	`simple_top`	トップモジュール。CPU+ メモリ +7 セグ表示 +I/O 接続。
`seg7dec.v`	`seg7dec`	7 セグメントデコーダ（導入課題から流用）。

5.2　ALU (`alu.v`)

16 ビット 2 入力の ALU で、加算・減算・論理演算・比較・転送の 7 演算を行う。減算は 2 の補数加算（`a + ~b + 1`）で実装し、CMP は SUB と同じ演算でフラグのみ設定する。

```
// alu.v - SIMPLE/B ALU
// ADD, SUB, AND, OR, XOR, CMP, MOV + アドレス計算(ADD流用)

module alu(
    input  [15:0] a,
    input  [15:0] b,
    input  [3:0]  op,
```

```
    output [15:0] result,
    output        flag_s,
    output        flag_z,
    output        flag_c,
    output        flag_v
);

wire [16:0] sum  = {1'b0, a} + {1'b0, b};
wire [16:0] diff = {1'b0, a} + {1'b0, ~b} + 17'd1;

assign result = (op == 4'h0) ? sum[15:0]  :  // ADD
                (op == 4'h1) ? diff[15:0]  :  // SUB
                (op == 4'h2) ? (a & b)     :  // AND
                (op == 4'h3) ? (a | b)     :  // OR
                (op == 4'h4) ? (a ^ b)     :  // XOR
                (op == 4'h5) ? diff[15:0]  :  // CMP (= SUB)
                (op == 4'h6) ? b           :  // MOV
                               16'h0000;

assign flag_s = result[15];
assign flag_z = (result == 16'h0000);

assign flag_c = (op == 4'h0)                ? sum[16]  :
                (op == 4'h1 || op == 4'h5)  ? diff[16] :
                                              1'b0;

wire add_ov = (~a[15] & ~b[15] & result[15]) | (a[15] & b[15] & ~result[15]);
wire sub_ov = (~a[15] &  b[15] & result[15]) | (a[15] & ~b[15] & ~result[15]);

assign flag_v = (op == 4'h0)                ? add_ov :
                (op == 4'h1 || op == 4'h5)  ? sub_ov :
                                              1'b0;

endmodule
```

Listing 1　alu.v

5.2.1　条件コード生成

- **S**: 演算結果の最上位ビット（`result[15]`）
- **Z**: 演算結果が 0 なら 1
- **C**: ADD では 17 ビット加算の桁上げ、SUB/CMP では 2 の補数加算の桁上げ（$a \geq b$ のとき 1）。論理演算・MOV では 0。
- **V**: ADD/SUB/CMP のみ。符号付きオーバーフロー検出。**（同符号の入力）** ∧ **（結果の符号が異なる）** で判定。

5.3 シフタ (shifter.v)

16 ビットデータを 0〜15 ビットシフトするバレルシフタ。4 種のシフト演算に対応する。

```
// shifter.v - SIMPLE/B バレルシフタ
// SLL(00), SLR(01), SRL(10), SRA(11)

module shifter(
    input  [15:0] data,
    input  [3:0]  d,
    input  [1:0]  op,
    output reg [15:0] result,
    output reg        flag_c
);

wire [16:0] sll_ext = {1'b0, data} << d;
wire [16:0] srl_ext = {data, 1'b0} >> d;

always @(*) begin
    case (op)
        2'b00: begin // SLL
            result = sll_ext[15:0];
            flag_c = (d == 4'd0) ? 1'b0 : sll_ext[16];
        end
        2'b01: begin // SLR (左循環シフト)
            result = (data << d) | (data >> (5'd16 - {1'b0, d}));
            flag_c = 1'b0;
        end
        2'b10: begin // SRL
            result = data >> d;
            flag_c = (d == 4'd0) ? 1'b0 : srl_ext[0];
        end
        2'b11: begin // SRA
            result = $signed(data) >>> d;
            flag_c = (d == 4'd0) ? 1'b0 : srl_ext[0];
        end
    endcase
end

endmodule
```

Listing 2　shifter.v

条件コード C は、シフト桁数が 0 または SLR（循環シフト）の場合は 0、それ以外では最後にシフトアウトされたビットの値が設定される。V は常に 0。

5.4 レジスタファイル (regfile.v)

8 本の 16 ビット汎用レジスタを提供する。2 つの読み出しポート（非同期、組み合わせ回路）と 1 つの書き込みポート（同期）を持つ。

```
// regfile.v - SIMPLE/B レジスタファイル (8 x 16bit)
// 2リード / 1ライトポート, 非同期リード

module regfile(
    input              clk,
    input              rst_n,
    input      [2:0]   rd_addr0,
    input      [2:0]   rd_addr1,
    output     [15:0]  rd_data0,
    output     [15:0]  rd_data1,
    input              we,
    input      [2:0]   wr_addr,
    input      [15:0]  wr_data
);

reg [15:0] regs [0:7];

assign rd_data0 = regs[rd_addr0];
assign rd_data1 = regs[rd_addr1];

integer i;
always @(posedge clk or negedge rst_n) begin
    if (~rst_n) begin
        for (i = 0; i < 8; i = i + 1)
            regs[i] <= 16'h0000;
    end else if (we) begin
        regs[wr_addr] <= wr_data;
    end
end

endmodule
```

Listing 3　regfile.v

非同期リードにより、p2 でレジスタ値が即座に AR/BR に供給される。書き込みは p5 で `rf_we` が有効な間にクロックエッジで実行される。

5.5 主記憶 (mem.v)

```
// mem.v - SIMPLE/B 主記憶
// 非同期リード / 同期ライト
// $readmemh でプログラムを初期化

```

```
module mem(
    input         clk,
    input  [15:0] addr,
    input  [15:0] wdata,
    input         we,
    output [15:0] rdata
);

parameter ADDR_WIDTH = 8;  // 256語（デフォルト）
parameter INIT_FILE  = "test.hex";

reg [15:0] ram [0:(1 << ADDR_WIDTH) - 1];

initial begin
    $readmemh(INIT_FILE, ram);
end

assign rdata = ram[addr[ADDR_WIDTH-1:0]];

always @(posedge clk) begin
    if (we)
        ram[addr[ADDR_WIDTH-1:0]] <= wdata;
end

endmodule
```

Listing 4　mem.v

非同期リード（assign 文）と同期ライト（always @(posedge clk)）を持つ。FPGA 上では Distributed RAM として実装される。$readmemh によりプログラムを初期化する。アドレス幅はパラメータで変更可能（デフォルト 8 ビット = 256 語）。

5.6　CPU コア (simple_cpu.v)

SIMPLE/B の全制御を行うモジュール。フェーズカウンタによる FSM とデータパス制御を含む。

```
// simple_cpu.v - SIMPLE/B CPU コア
// 5フェーズ順次実行: p1(IF) → p2(RR) → p3(EX) → p4(MA) → p5(WB)
//
// ブロック図（SIMPLE設計資料 図2 準拠):
//   PC → メモリ → IR → デコード → レジスタファイル(AR,BR)
//   → ALU/シフタ → DR → メモリ/IO → MDR → レジスタ書き戻し

module simple_cpu(
    input         clk,
    input         rst_n,
    input         exec,
    // メモリインタフェース
    output [15:0] mem_addr,
    input  [15:0] mem_rdata,
```

```
    output [15:0] mem_wdata,
    output        mem_we,
    // I/O インタフェース
    input  [15:0] in_data,
    output reg [15:0] out_data,
    output reg        out_we,
    // ステータス
    output        halted
);

// フェーズ定義
localparam P1 = 3'd0; // 命令フェッチ
localparam P2 = 3'd1; // レジスタ読み出し
localparam P3 = 3'd2; // 演算
localparam P4 = 3'd3; // 主記憶アクセス
localparam P5 = 3'd4; // レジスタ書き込み

// 内部レジスタ
reg [15:0] pc, ir, ar, br, dr, mdr;
reg [2:0]  phase;
reg        flag_s, flag_z, flag_c, flag_v;
reg        running;

// ---- 命令デコード ----
wire [1:0] op1   = ir[15:14];
wire [2:0] rs_ra = ir[13:11]; // Rs（演算）/ Ra（LD/ST）
wire [2:0] rd_rb = ir[10:8];  // Rd（演算）/ Rb（LD/ST/LI）
wire [3:0] op3   = ir[7:4];   // 演算/IO命令コード
wire [3:0] d4    = ir[3:0];   // シフト桁数
wire [7:0] d8    = ir[7:0];   // 即値/変位
wire [2:0] op2   = ir[13:11]; // LI/B 命令コード
wire [2:0] cond  = ir[10:8];  // 条件分岐条件

wire [15:0] sext_d8 = {{8{d8[7]}}, d8};

// 命令種別判定
wire is_alu   = (op1 == 2'b11);
wire is_load  = (op1 == 2'b00);
wire is_store = (op1 == 2'b01);
wire is_li    = (op1 == 2'b10) && (op2 == 3'b000);
wire is_b     = (op1 == 2'b10) && (op2 == 3'b100);
wire is_bcc   = (op1 == 2'b10) && (op2 == 3'b111);

wire is_shift  = is_alu && op3[3] && ~op3[2];           // 1000-1011
wire is_in     = is_alu && (op3 == 4'b1100);
wire is_out    = is_alu && (op3 == 4'b1101);
wire is_hlt    = is_alu && (op3 == 4'b1111);
wire is_alu_rr = is_alu && ~op3[3] &&
                 (op3 != 4'b0111);                       // 0000-0110 (
```

```
                    reserved除く)

// ---- レジスタファイル ----
wire [15:0] rf_rd0, rf_rd1;

wire rf_we_need = (is_alu_rr && op3 != 4'b0101) ||        // ALU (CMP除く)
                  is_shift || is_load || is_li || is_in;
wire        rf_we      = (phase == P5) && running && rf_we_need;
wire [2:0]  rf_wr_addr = is_load ? rs_ra : rd_rb;
wire [15:0] rf_wr_data = (is_load || is_in) ? mdr : dr;

regfile rf(
    .clk(clk), .rst_n(rst_n),
    .rd_addr0(rs_ra), .rd_data0(rf_rd0),
    .rd_addr1(rd_rb), .rd_data1(rf_rd1),
    .we(rf_we), .wr_addr(rf_wr_addr), .wr_data(rf_wr_data)
);

// ---- ALU ----
wire [15:0] alu_a = is_li           ? 16'h0000 :
                    (is_b || is_bcc) ? pc       :
                                       br;
wire [15:0] alu_b = is_alu ? ar : sext_d8;
wire [3:0]  alu_op = is_alu ? op3 : 4'h0; // 非ALU命令はADD

wire [15:0] alu_result;
wire        alu_s, alu_z, alu_c, alu_v;

alu alu_inst(
    .a(alu_a), .b(alu_b), .op(alu_op),
    .result(alu_result),
    .flag_s(alu_s), .flag_z(alu_z), .flag_c(alu_c), .flag_v(alu_v)
);

// ---- シフタ ----
wire [15:0] shift_result;
wire        shift_c;

shifter shift_inst(
    .data(br), .d(d4), .op(op3[1:0]),
    .result(shift_result), .flag_c(shift_c)
);

// ---- メモリインタフェース (組み合わせ) ----
assign mem_addr  = (phase == P1) ? pc : dr;
assign mem_wdata = ar;
assign mem_we    = (phase == P4) && running && is_store;

// ---- 分岐条件判定 ----
```

```
reg branch_taken;
always @(*) begin
    case (cond)
        3'b000:  branch_taken = flag_z;                      // BE
        3'b001:  branch_taken = flag_s ^ flag_v;             // BLT
        3'b010:  branch_taken = flag_z | (flag_s ^ flag_v);  // BLE
        3'b011:  branch_taken = ~flag_z;                     // BNE
        default: branch_taken = 1'b0;
    endcase
end

// ---- ステータス出力 ----
assign halted = ~running;

// ---- exec 立ち上がり検出 ----
reg exec_prev;
always @(posedge clk or negedge rst_n) begin
    if (~rst_n) exec_prev <= 1'b0;
    else        exec_prev <= exec;
end
wire exec_rise = exec & ~exec_prev;

// ---- メインステートマシン ----
always @(posedge clk or negedge rst_n) begin
    if (~rst_n) begin
        pc       <= 16'h0000;
        ir       <= 16'h0000;
        ar       <= 16'h0000;
        br       <= 16'h0000;
        dr       <= 16'h0000;
        mdr      <= 16'h0000;
        phase    <= P1;
        flag_s   <= 1'b0;
        flag_z   <= 1'b0;
        flag_c   <= 1'b0;
        flag_v   <= 1'b0;
        running  <= 1'b0;
        out_data <= 16'h0000;
        out_we   <= 1'b0;
    end else begin
        out_we <= 1'b0;

        if (~running && exec_rise)
            running <= 1'b1;

        if (running) begin
            case (phase)
                // ---- p1: 命令フェッチ ----
                P1: begin
```

```
                    ir    <= mem_rdata;
                    pc    <= pc + 16'd1;
                    phase <= P2;
                end

                // ---- p2: レジスタ読み出し ----
                P2: begin
                    ar    <= rf_rd0; // r[Rs/Ra]
                    br    <= rf_rd1; // r[Rd/Rb]
                    phase <= P3;
                end

                // ---- p3: 演算 ----
                P3: begin
                    if (is_shift)
                        dr <= shift_result;
                    else
                        dr <= alu_result;

                    if (is_alu_rr) begin
                        flag_s <= alu_s;
                        flag_z <= alu_z;
                        flag_c <= alu_c;
                        flag_v <= alu_v;
                    end else if (is_shift) begin
                        flag_s <= shift_result[15];
                        flag_z <= (shift_result == 16'h0000);
                        flag_c <= shift_c;
                        flag_v <= 1'b0;
                    end

                    phase <= P4;
                end

                // ---- p4: 主記憶アクセス / 入出力 ----
                P4: begin
                    if (is_load)
                        mdr <= mem_rdata;
                    if (is_in)
                        mdr <= in_data;
                    if (is_out) begin
                        out_data <= ar;
                        out_we   <= 1'b1;
                    end
                    phase <= P5;
                end

                // ---- p5: レジスタ書き込み / 分岐 ----
                P5: begin
```

```
                    if (is_b)
                        pc <= dr;
                    if (is_bcc && branch_taken)
                        pc <= dr;
                    if (is_hlt)
                        running <= 1'b0;
                    phase <= P1;
                end
            endcase
        end
    end
end

endmodule
```

Listing 5　simple_cpu.v

5.6.1　命令デコード

IR の上位 2 ビット（op1）で命令形式を判別し、演算命令では op3 フィールドで具体的な演算を特定する。デコード結果は組み合わせ回路として生成され、各フェーズの制御に使用される。

5.6.2　ALU 入力マルチプレクサ

ALU 入力 A は命令種別に応じて以下から選択される:

- ALU 演算・LD/ST: BR（r[Rd] または r[Rb]）
- LI: 定数 0（即値をそのまま通すため）
- B/Bcc: PC（分岐先アドレス計算のため）

ALU 入力 B は:

- ALU 演算: AR（r[Rs]）
- LD/ST/LI/B/Bcc: sign_ext(d8)（8 ビット即値/変位の符号拡張）

5.6.3　レジスタ書き込み制御

書き込みイネーブル信号 rf_we は組み合わせ論理として生成される。これにより、p5 のフェーズ中に rf_we = 1 となり、p5→p1 のクロックエッジでレジスタファイルへの書き込みが実行される。

5.7　トップモジュール (simple_top.v)

CPU、メモリ、7 セグメント表示を統合するトップモジュール。OUT 命令で出力された 16 ビット値をラッチし、ダイナミック点灯により 4 桁の 16 進数として 7 セグメント LED に表示する。

```
// simple_top.v - SIMPLE/B トップモジュール
// CPU + メモリ + 7セグ表示 + I/O
//
// ピンアサイン: ピンアサイン表を参照して設定すること
```

```
//    clk       : 20MHz クロック
//    rst_n     : リセットボタン（負論理）
//    exec      : 実行ボタン（正論理: 押すと1）
//    sw[15:0]  : DIPスイッチ（IN命令用，負論理）
//    seg[7:0]  : 7セグメントLEDセグメント出力
//    sel[7:0]  : 7セグメントLED桁選択

module simple_top(
    input         clk,
    input         rst_n,
    input         exec,
    input  [15:0] sw,
    output [7:0]  seg,
    output [7:0]  sel
);

// ---- CPU ↔ メモリ接続 ----
wire [15:0] mem_addr, mem_rdata, mem_wdata;
wire        mem_we;

// ---- CPU ↔ I/O ----
wire [15:0] cpu_out_data;
wire        cpu_out_we;
wire        cpu_halted;

simple_cpu cpu(
    .clk(clk), .rst_n(rst_n), .exec(exec),
    .mem_addr(mem_addr), .mem_rdata(mem_rdata),
    .mem_wdata(mem_wdata), .mem_we(mem_we),
    .in_data(~sw),  // DIPスイッチ負論理を反転
    .out_data(cpu_out_data), .out_we(cpu_out_we),
    .halted(cpu_halted)
);

mem #(.ADDR_WIDTH(8), .INIT_FILE("test.hex")) memory(
    .clk(clk),
    .addr(mem_addr), .rdata(mem_rdata),
    .wdata(mem_wdata), .we(mem_we)
);

// ---- OUT レジスタ ----
reg [15:0] out_reg;
always @(posedge clk or negedge rst_n) begin
    if (~rst_n)
        out_reg <= 16'h0000;
    else if (cpu_out_we)
        out_reg <= cpu_out_data;
end

```

```
// ---- 7セグ表示（out_reg を4桁16進数で表示）----
parameter DISP_DIV = 5000; // 20MHz / 5000 = 4kHz

reg [12:0] disp_cnt;
wire       disp_en = (disp_cnt == DISP_DIV - 1);

always @(posedge clk or negedge rst_n) begin
    if (~rst_n)      disp_cnt <= 13'd0;
    else if (disp_en) disp_cnt <= 13'd0;
    else             disp_cnt <= disp_cnt + 13'd1;
end

reg [1:0] dig_sel;
always @(posedge clk or negedge rst_n) begin
    if (~rst_n)       dig_sel <= 2'd0;
    else if (disp_en) dig_sel <= dig_sel + 2'd1;
end

wire [3:0] cur_dig = (dig_sel == 2'd0) ? out_reg[3:0]   :
                     (dig_sel == 2'd1) ? out_reg[7:4]   :
                     (dig_sel == 2'd2) ? out_reg[11:8]  :
                                         out_reg[15:12] ;

seg7dec dec(.din(cur_dig), .seg(seg));

assign sel = (dig_sel == 2'd0) ? 8'b11111110 :
             (dig_sel == 2'd1) ? 8'b11111101 :
             (dig_sel == 2'd2) ? 8'b11111011 :
                                 8'b11110111 ;

endmodule
```

Listing 6　simple_top.v

6　テストプログラム

6.1　基本テスト: 加算と出力

最も単純なテストとして、即値ロード・加算・出力・停止を行うプログラムを使用した。

6.2　応用テスト: 遅延付きカウンタ

実機デモ用として、遅延ループ付きのインクリメントカウンタを作成した。内側ループ（65,536 回 × 2 命令）× 外側ループ（10 回）で約 0.3 秒の遅延を実現し、カウンタ値を OUT 命令で 7 セグメント LED に出力する。

表 7　基本テストプログラム

アドレス	機械語	ニーモニック	動作	期待値
0	0x8003	LI r0, 3	r[0] = 3	r0 = 3
1	0x8105	LI r1, 5	r[1] = 5	r1 = 5
2	0xC800	ADD r0, r1	r[0] = r[0] + r[1]	r0 = 8
3	0xC0D0	OUT r0	output = r[0]	OUT = 8
4	0xC0F0	HLT	停止	halted

表 8　遅延付きカウンタプログラム

アドレス	機械語	ニーモニック	動作
0	0x8000	LI r0, 0	カウンタ初期化
1	0x8101	LI r1, 1	インクリメント値
2	0x840A	LI r4, 10	外側ループ回数
3	0xC800	ADD r0, r1	r0++
4	0xC0D0	OUT r0	出力
5	0x8300	LI r3, 0	外側カウンタ初期化
6	0x8200	LI r2, 0	内側カウンタ初期化
7	0xCA00	ADD r2, r1	r2++
8	0xBBFE	BNE -2	r2≠0 なら addr 7 へ
9	0xCB00	ADD r3, r1	r3++
10	0xE350	CMP r3, r4	r3 と 10 を比較
11	0xB9FA	BLT -6	r3 < 10 なら addr 6 へ
12	0xA0F6	B -10	addr 3 へ戻る

6.2.1　遅延時間の計算

1 命令あたり 5 クロックサイクル、クロック周波数 20MHz で計算する。

- 内側ループ: $65{,}536 \times 2 \times 5 = 655{,}360$ サイクル
- 外側ループ: $10 \times (655{,}360 + \alpha) \approx 6{,}600{,}000$ サイクル
- 所要時間: $6{,}600{,}000 \div 20{,}000{,}000 \approx 0.33$ 秒/カウント

約 3 回/秒のペースでカウンタが更新され、7 セグメント LED で目視確認可能な速度となる。

7　動作検証

7.1　テストベンチ

```
// simple_tb.v - SIMPLE/B テストベンチ
// テストプログラムを実行し、OUT命令の出力と HLT による停止を検証

`timescale 1ns / 1ps

```

```
module simple_tb;

reg          clk;
reg          rst_n;
reg          exec;
wire [15:0] mem_addr, mem_rdata, mem_wdata;
wire         mem_we;
wire [15:0] out_data;
wire         out_we;
wire         halted;

// ---- CPU ----
simple_cpu cpu(
    .clk(clk), .rst_n(rst_n), .exec(exec),
    .mem_addr(mem_addr), .mem_rdata(mem_rdata),
    .mem_wdata(mem_wdata), .mem_we(mem_we),
    .in_data(16'h00FF),
    .out_data(out_data), .out_we(out_we),
    .halted(halted)
);

// ---- メモリ（テスト用小プログラム）----
// LI r0,3 / LI r1,5 / ADD r0,r1 / OUT r0 / HLT
reg [15:0] ram [0:255];
initial begin
    ram[0] = 16'h8003; // LI r0, 3
    ram[1] = 16'h8105; // LI r1, 5
    ram[2] = 16'hC800; // ADD r0, r1
    ram[3] = 16'hC0D0; // OUT r0
    ram[4] = 16'hC0F0; // HLT
end

assign mem_rdata = ram[mem_addr[7:0]];
always @(posedge clk) begin
    if (mem_we) ram[mem_addr[7:0]] <= mem_wdata;
end

// ---- クロック（20MHz: 周期50ns）----
always #25 clk = ~clk;

// ---- 出力監視 ----
always @(posedge clk) begin
    if (out_we)
        $display("Time %0t: OUT = %h (%0d)", $time, out_data, out_data);
end

// ---- テストシナリオ ----
initial begin
    clk   = 1'b0;
```

```
    rst_n = 1'b1;
    exec  = 1'b0;

    // リセット
    #50  rst_n = 1'b0;
    #100 rst_n = 1'b1;
    #50;

    // 実行開始
    $display("--- Execution Start ---");
    exec = 1'b1;
    #50;
    exec = 1'b0;

    // HLT まで待機（最大200サイクル）
    begin : wait_halt
        integer i;
        for (i = 0; i < 200; i = i + 1) begin
            @(posedge clk);
            if (halted) begin
                $display("--- CPU Halted (PC=%h) ---", cpu.pc);
                $finish;
            end
        end
    end

    $display("--- Timeout ---");
    $finish;
end

endmodule
```

Listing 7　simple_tb.v — テストベンチ

テストベンチは CPU とメモリをインスタンス化し、基本テストプログラムを内部メモリに格納して実行する。OUT 命令の出力を`$display`で監視し、HLT による停止を halted 信号で検出する。

7.2　シミュレーション結果

Icarus Verilog 11.0 によるシミュレーション結果を以下に示す。

```
--- Execution Start ---
Time 1225000: OUT = 0008 (8)
--- CPU Halted (PC=0005) ---
```

Listing 8　シミュレーション出力

7.2.1　結果の検証

- **OUT = 0008**: LI r0,3 + LI r1,5 + ADD r0,r1 = 3 + 5 = 8。期待通りの結果。

- **PC = 0005**: HLT 命令（アドレス 4）の実行後、PC は 4+1=5。正しい値。
- **タイミング**: 5 命令 × 5 フェーズ × 50ns/サイクル = 1,250ns。出力タイミング（Time 1225000ps = 1,225ns）はリセット解除後の遅延と exec 検出を考慮すると妥当。

7.3 命令実行トレース

各命令のフェーズごとの動作を表 9 にまとめる。

表 9　命令実行トレース

命令	フェーズ	主要な動作
`LI r0, 3`	p1	IR←0x8003, PC←1
	p3	DR←ALU(0, sext(3), ADD) = 3
	p5	r[0]←DR = 3
`LI r1, 5`	p1	IR←0x8105, PC←2
	p3	DR←ALU(0, sext(5), ADD) = 5
	p5	r[1]←DR = 5
`ADD r0, r1`	p1	IR←0xC800, PC←3
	p2	AR←r[1]=5, BR←r[0]=3
	p3	DR←ALU(3, 5, ADD) = 8; S=0, Z=0, C=0, V=0
	p5	r[0]←DR = 8
`OUT r0`	p2	AR←r[0]=8
	p4	output←AR = 8
`HLT`	p5	running←0

8 性能評価

8.1 命令実行サイクル数

SIMPLE/B では全命令が 5 サイクルで実行される。20MHz クロックでは 1 命令あたり 250ns であり、スループットは 4 MIPS (Million Instructions Per Second) となる。

8.2 回路規模の見積もり

- **レジスタファイル**: 8 × 16 = 128 ビットのフリップフロップ
- **ALU**: 16 ビット加算器 × 1、各種論理演算のマルチプレクサ
- **シフタ**: 16 ビットバレルシフタ（4 段マルチプレクサ）
- **制御部**: フェーズカウンタ（3 ビット）、命令デコーダ（組み合わせ回路）
- **内部レジスタ**: PC, IR, AR, BR, DR, MDR（各 16 ビット）+ SZCV（4 ビット）
- **主記憶**: 256 語 × 16 ビット = 4,096 ビット（Distributed RAM）

Quartus Prime でのコンパイル後、Fitter → Summary で具体的な LE 数を確認すること。

8.3 最大動作周波数

クリティカルパスは以下の経路が候補となる:

- p1: PC → メモリアドレス → メモリ読み出し → IR ラッチ
- p3: BR/AR → ALU 演算 → DR ラッチ
- p4: DR → メモリアドレス → メモリ読み出し → MDR ラッチ

非同期リードメモリ（Distributed RAM）を使用しているため、メモリアクセスを含むパスが最も長くなる可能性が高い。Timing Analyzer で Fmax を確認し、20MHz 以上であることを確認すること。

8.4 改良の方向性

SIMPLE/B は最も基本的な設計であり、以下の改良が考えられる。

- **フェーズの並列実行**: p1/p5 の並列実行で 4 サイクル化、さらに p1/p3 と p2/p5 の並列実行で 3 サイクル化が可能。
- **パイプライン化**: 全フェーズを並列実行し、理想的には 1 サイクル/命令（5 倍の性能向上）を目指す。データハザードやコントロールハザードの対策が必要。
- **Block RAM の利用**: 同期リードの Block RAM に切り替えることで、より大きなメモリ空間を使用可能。ただしメモリアクセスのタイミング調整が必要。

9 まとめ

本レポートでは、SIMPLE アーキテクチャに準拠した 16 ビットプロセッサ SIMPLE/B を Verilog HDL で設計・実装し、シミュレーションにより動作を検証した。

- 5 フェーズ順次実行方式（IF → RR → EX → MA → WB）により、全命令を統一的に 5 サイクルで実行する構成とした。
- ALU（7 演算 + フラグ生成）、バレルシフタ（4 種シフト）、レジスタファイル（8×16 ビット）を独立モジュールとして設計し、CPU コアから接続する階層構造とした。
- 基本テストプログラム（LI/ADD/OUT/HLT）のシミュレーションにより、即値ロード・加算・出力・停止の正常動作を確認した（OUT = 8）。
- 遅延付きカウンタプログラムにより、LD/ST 以外の主要命令（LI, ADD, CMP, B, BNE, BLT, OUT）の動作を検証可能とした。

今後は実機（MU500-RK）への実装と動作確認、およびパイプライン化等のマイクロアーキテクチャ改良を進める予定である。